Software Requirements Specification

for

AI-Powered Air-Gesture Authentication System ("Sci-Fi" Hand Tracking)

Version 1.0 approved

Prepared by SANTHOSH S,Ronald Packian,Jai Vishwa

Mepco Schlenk Engineering College,Sivakasi

Date:23.07.2026

Table of Contents

Table of Contents ..... **ii**
Revision History ..... **ii**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 2
2. Overall Description **2**
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 3
2.5 Design and Implementation Constraints 3
2.6 User Documentation 3
2.7 Assumptions and Dependencies 3
3. External Interface Requirements **4**
3.1 User Interfaces 4
3.2 Hardware Interfaces 4
3.3 Software Interfaces 4
3.4 Communications Interfaces 4
4. System Features **5**
4.1 System Feature 1 5
4.2 System Feature 2 (and so on) ..... **Error! Bookmark not defined.**
5. Other Nonfunctional Requirements **7**
5.1 Performance Requirements 7
5.2 Safety Requirements 7
5.3 Security Requirements 7
5.4 Software Quality Attributes 7
5.5 Business Rules 8
6. Other Requirements **8**
Appendix A: Glossary **8**
Appendix B: Analysis Models ..... **Error! Bookmark not defined.**
Appendix C: To Be Determined List ..... **Error! Bookmark not defined.**

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for the AI-Powered Air-Gesture Authentication System, version 1.0. This SRS covers the entire scope of the web-based application, including the React frontend UI, the client-side AI hand-tracking engine, and the NoSQL backend data management system.

1.2 Document Conventions

Requirements are uniquely identified with the prefix REQ- followed by a sequence number. High-level features dictate the priority of their sub-requirements. Italics are used to highlight specific UI elements or database collections.

1.3 Intended Audience and Reading Suggestions

This document is intended for front-end developers, AI/machine learning engineers, backend database administrators, project managers, and quality assurance testers. Readers should begin with Section 2 (Overall Description) to understand the system context before proceeding to Section 4 (System Features) for specific functional requirements.

1.4 Product Scope

The AI-Powered Air-Gesture Authentication System is a web-based, contactless biometric security portal. It replaces traditional text-based passwords with spatial hand gestures (e.g., a pinch or a peace sign). By utilizing client-side AI (TensorFlow.js/MediaPipe) to track hand movements and a NoSQL database to store complex 3D gesture arrays, the system provides a highly secure, hygienic, and accessible login experience with a futuristic "Sci-Fi" visual interface.

1.5 References

- MediaPipe Handpose Documentation (Google Developers)
- React.js Official Documentation
- MongoDB / Firebase NoSQL Data Modeling Guidelines

2. Overall Description

2.1 Product Perspective

This system is a new, self-contained web application that acts as an authentication portal. It can function as a standalone secure dashboard or serve as an authentication microservice module integrated into larger enterprise applications. It relies on standard device webcams and does not require external biometric hardware.

2.2 Product Functions

User Registration & Gesture Enrollment: Capture and store a baseline 3D mathematical representation of a user's chosen hand gesture.Biometric Authentication: Compare real-time hand movements against the stored baseline to grant or deny access.Real-Time Skeletal Rendering: Display a live, glowing 21-point hand skeleton over the user's video feed.Security Auditing: Log all access attempts, timestamps, and AI confidence scores.

2.3 User Classes and Characteristics

Standard Users: General users who use the system to log into their dashboard. May include users with motor impairments who find keyboards difficult to use.Administrators: IT or security personnel who monitor system access logs and manage user accounts via the Admin Dashboard.

2.4 Operating Environment

Client Side: Modern web browsers (Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge) with HTML5 Canvas and WebGL support.Hardware Requirement: Standard 720p or 1080p integrated/external webcam. Server Side/Backend: Node.js environment interacting with a NoSQL database (e.g., MongoDB or Firebase Firestore).

2.5 Design and Implementation Constraints

Privacy Constraint: All video processing and frame analysis must occur client-side in the browser. No raw video feeds or images can be transmitted to or stored on the server.

Protocol: The application must operate over HTTPS to access the browser's MediaDevices.getUserMedia() API.

Data Structure: Gesture data must be stored in a NoSQL environment due to the variable length and hierarchical nature of 3D spatial coordinate arrays.

2.6 User Documentation

- On-screen tooltips explaining how to position the hand for the camera.
- A digital user manual detailing how to reset a biometric gesture if forgotten.

2.7 Assumptions and Dependencies

Users are operating in an environment with adequate lighting for the webcam to detect hand features.The system depends on the ongoing availability and maintenance of the open-source MediaPipe/TensorFlow.js libraries.

3. External Interface Requirements

3.1 User Interfaces

- The UI will be built using React.js.
- **Login Screen:** Shall feature a central video player overlaid with a transparent HTML5 Canvas. A futuristic, dark-mode color palette (neon blues/greens on black) will be used.
- **Visual Feedback:** The canvas will draw lines connecting the 21 finger joints. The lines will turn Green for a successful match and Red for a failed match.

3.2 Hardware Interfaces

The system interfaces directly with the user's local webcam via standard browser APIs. No specialized infrared or depth-sensing cameras are required.

3.3 Software Interfaces

NoSQL Database Interface: The system will communicate with a NoSQL database via REST APIs or proprietary SDKs (e.g., Firebase SDK) to push and pull JSON documents.
AI Library Interface: The application will invoke the hand-pose-detection module from TensorFlow.js to process video frames.

3.4 Communications Interfaces

All client-server communications will be encrypted using TLS/SSL (HTTPS).Data payloads (gesture coordinate arrays and authentication tokens) will be formatted as JSON.

4. System Features

4.1 System Feature 1

4.1.1 Description and Priority

Priority: High. Users must be able to record a custom hand gesture to act as their biometric password.

4.1.2 Stimulus/Response Sequences

Stimulus: User clicks "Register New Gesture" and holds a specific hand sign in front of the camera for 3 seconds.

Response: System captures the 3D coordinates, averages them to create a baseline, and saves the array to the NoSQL database.

4.2 Feature 2: Real-Time Biometric Authentication

4.2.1 Description and Priority

Priority: High. The core security mechanism that validates a user based on their hand gesture.

REQ-1: The system shall prompt the user to hold their hand steady within a designated on-screen frame.

REQ-2: The system shall extract the X, Y, and Z coordinates of the 21 hand landmarks.

REQ-3: The system shall serialize these coordinates into a JSON array and store them in the NoSQL database under the user's profile document.

4.2.2 Stimulus/Response Sequences

Stimulus: An unregistered/logged-out user performs a hand gesture in front of the webcam.

Response: The system calculates the similarity to the stored gesture. If the confidence score is > 85%, access is granted.

4.2.3 Functional Requirements

REQ-4: The client-side AI shall process the video feed at a minimum of 15 frames per second (FPS).

REQ-5: The system shall apply a mathematical algorithm (e.g., Cosine Similarity or Euclidean Distance) to compare the live gesture vector with the stored baseline vector.

REQ-6: If the match score exceeds the defined threshold, the system shall issue a secure JWT (JSON Web Token) and route the user to the dashboard.

4.3 Feature 3: Interactive Sci-Fi Skeletal Overlay

4.3.1 Description and Priority

Priority: Medium. Visual feedback mechanism to enhance user experience and "wow" factor.

4.3.2 Stimulus/Response Sequences

Stimulus: The webcam detects a hand in the frame.

Response: A neon skeleton is dynamically drawn over the user's hand, following its movements in real-time.

4.3.3 Functional Requirements

REQ-7: The React application shall overlay a transparent <canvas> precisely on top of the <video> element.

REQ-8: The system shall render lines and nodes connecting the 21 hand landmarks provided by the AI model.

4.4 Feature 4: Security Access Logging

4.4.1 Description and Priority

Priority: Low. Administrative feature for tracking usage and security events.

4.4.2 Stimulus/Response Sequences

Stimulus: A user attempts to authenticate (success or failure).

Response: A log entry is created in the NoSQL database.

4.4.3 Functional Requirements

REQ-9: The system shall record a time-stamped document in the NoSQL database for every login attempt, including the AI confidence score.

REQ-10: The system shall lock the user interface for 60 seconds after 3 consecutive failed gesture attempts.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The AI hand-tracking model must initialize within 3 seconds of the user granting camera permissions.Gesture validation math must execute in under 500 milliseconds to provide a seamless login experience.

5.2 Safety Requirements

The system must not require users to perform physically strenuous or uncomfortable hand contortions. Gestures must be ergonomically safe.

5.3 Security Requirements

Strict Privacy: Under no circumstances will video frames be sent over the network. All AI processing is strictly client-side.Database access must be restricted to authenticated backend services only. No direct client-to-database writes are permitted for enrollment without proper authentication tokens.

5.4 Software Quality Attributes

Accessibility: The system provides an alternative for users unable to utilize traditional keyboards (motor impairment).
Usability: The visual feedback (glowing skeleton) must be highly responsive to prevent user confusion

5.5 Business Rules

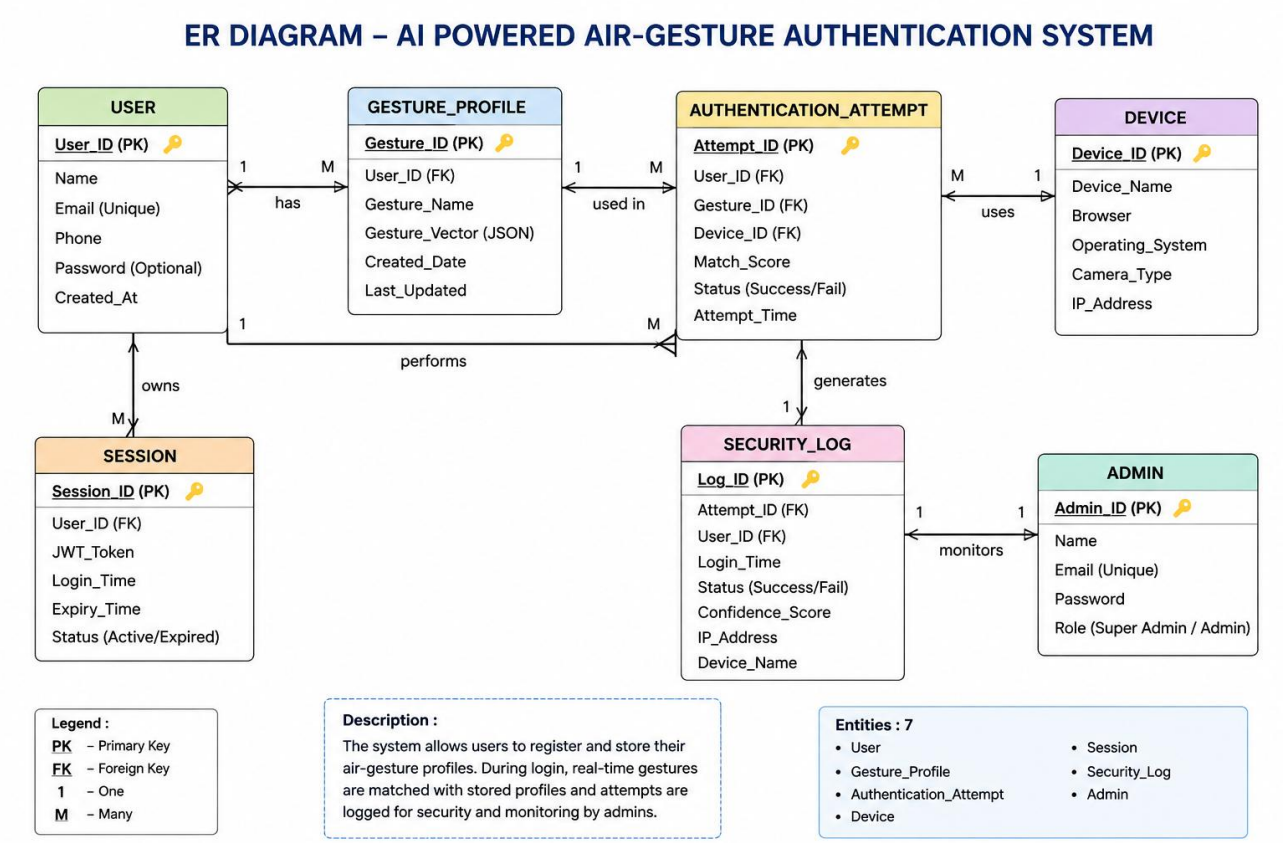
A user may only have one active biometric gesture registered at any given time. Gestures must contain at least 2 distinct finger positions (e.g., a flat, open palm is too generic and will be rejected during enrollment).

6. Other Requirements

Database Requirements (NoSQL Schema Constraint):

The database must support dynamic, nested JSON documents. A standard relational (SQL) database is prohibited for biometric storage due to the complex, multi-dimensional nature of the 21-point X/Y/Z coordinate arrays.

7. ER Diagram



Appendix A: Glossary

AI/ML: Artificial Intelligence / Machine Learning.

MediaPipe: An open-source framework by Google for building cross-platform, customizable machine learning solutions for live media.

NoSQL: A non-relational database design that allows for the storage of flexible, unstructured, or semi-structured data (e.g., JSON).

JWT (JSON Web Token): An open standard used to share security information between two parties securely.

X/Y/Z Coordinates: Spatial data points representing horizontal (X), vertical (Y), and depth (Z) positions of hand joints.

Appendix B: Analysis Models

(In a final document, insert a diagram here showing the Data Flow: Webcam -> Client-Side React Canvas -> MediaPipe AI Model -> Match Algorithm -> NoSQL DB)

Appendix C: To Be Determined List

TBD: The exact mathematical distance threshold for gesture acceptance (currently estimated at 85%, requires user testing).

TBD: Fallback authentication method if the user's webcam is broken.